Software Project Management – IS212

Agentic AI and the Software Team

Announcements

Announcements – CFO's Request

- ConnectSphere's CFO has discovered AI and has a question for your team!
- Please see the update in eLearn, and consider his request in advance of the project submission
- **Think about how you will respond**
- We will ask for your response in the slide deck deliverable for the project (template released in Week 11)



Announcements – Week 11 Quiz

- **Week 9 Quiz results are out –** *please ask us if any doubts*
  - *Of the 4 quizzes, only your top 3 quizzes retained*
- Our final quiz will take place at the start of your Week 11 class
  - *Starts promptly at the beginning of class!*
  - *No extra time if you arrive to class late*
- Need to be physically present (no make-ups; see Week 1 slides)
- Requires Respondus LockDown Browser
- Short (~10 mins); covers the Weeks 9–10 official materials
  - *MCQ-style questions (no written)*

Recap

Week 9 Recap – Automated Testing #2; CD

- You stub the payment gateway and every unit test passes, yet production charges 50 cents instead of $50. Why didn't the tests catch it, and what kind of test would?



- You need to check that exactly one receipt is sent. Stub or mock... and why?

- A test fails one run in ten on the same commit. What could cause this?

- Continuous delivery vs continuous deployment: what's the difference, and who decides when a change is released?

Today: Agentic AI and the Software Team

Objectives

On completing this class, you will be able to:

- **Explain** how delegating work to coding agents changes a team's **roles**, **coordination** and **bottlenecks** – using Scrum concepts from earlier weeks
- **Distinguish** guidance (instruction files, style guides, project "constitutions") from enforcement (tests, CI gates, branch protection, permissions)
- **Evaluate** an agent-authored change as a reviewer, and recognise agent-era forms of technical debt
- **Decide** how much autonomy to give an agent for a task, based on reversibility, blast radius and verifiability

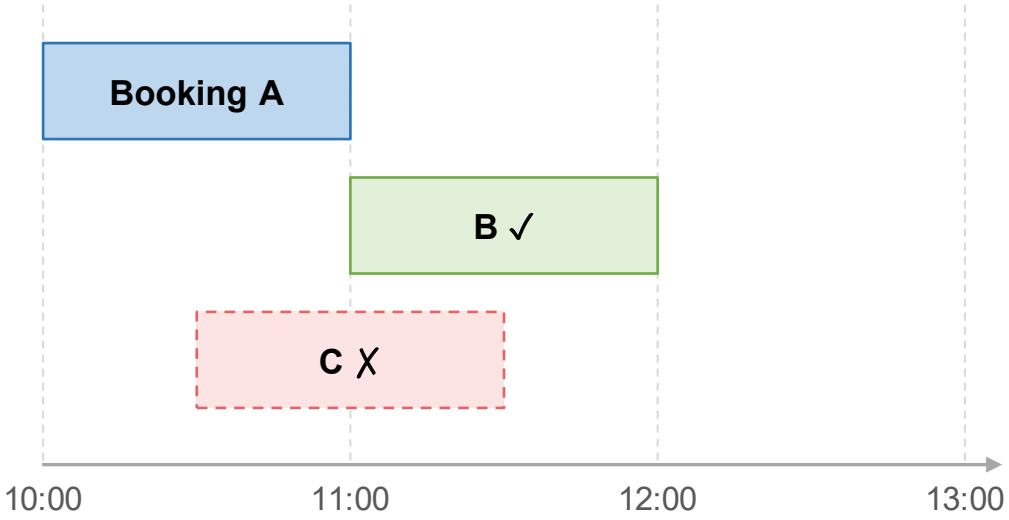
Outline

- **Part 1 – What are we delegating?** – the agentic loop; delegates vs teammates
- **Part 2 – Making delegation safe** – the system around the agent; guidance vs enforcement; demo
- **Part 3 – Review, accountability, debt** – the review bottleneck; agentic-era debt; a worked PR example
- **Part 4 – Delegation by risk** – reversibility, blast radius, verifiability (delegation dial exercise)

Running Example – Study-Room Booking

1. A booking lasts **at most 2 hours**
2. Rooms can be booked **up to 7 days ahead**
3. **Back-to-back** bookings are fine; **overlapping** ones are not
4. Cancelling **less than 2 h before the start** records a late-cancellation strike

Rule #3 on one room – GSR 2-01, 20 Oct



B: back-to-back with A – allowed. C: overlaps A – rejected.

Monday Morning

Pull requests · 23 Open · all checks passing
✓ Allow students to extend an existing booking #57 · opened by agent
✓ Add a 'my bookings' page #58 · opened by agent
✓ Refactor date handling across services #59 · opened by agent
✓ Bump Flask and other dependencies #60 · opened by agent
✓ Record late-cancellation strikes #61 · opened by agent
✓ Email a reminder 1 h before each booking #62 · opened by agent
✓ Update tests for new error format #63 · opened by agent
… and 16 more



Your team is building the **campus study-room booking app**

On Friday, four of you each left a coding agent working through backlog items

Monday, 9 am:
23 open pull requests · ~9,000 changed lines · **every check green**

Sprint Review is on **Wednesday** ⚠

What do you do first?

A Merge everything that's green
B Review all 23, oldest first
C …or something else – **what?**

The Question for Today

Agentic development changes the **management problem**

From:

How do we get enough software built?

Towards:

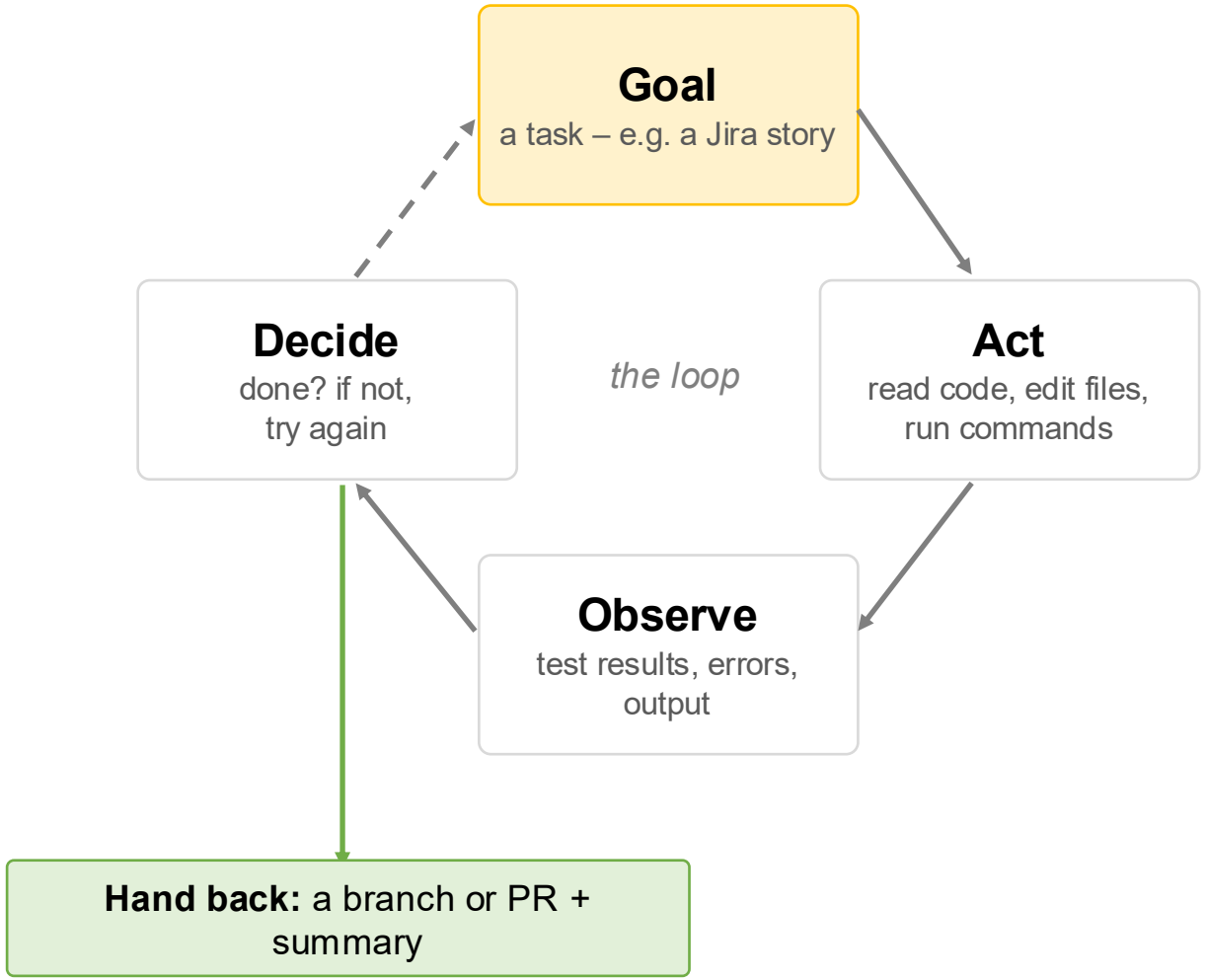
Delegate	What should we delegate? Part 1
Constrain	What should constrain it? Part 2
Verify	How do we verify the result? Part 3
Own	Who owns the decision? Part 4

What are we Delegating?

Part 1 of 4

delegate constrain verify own

What Makes an AI Tool 'Agentic'?



A widely used working definition:

An LLM agent runs **tools** in a **loop** to achieve a **goal**.

- **Tools** = files, terminal, tests, browser, issue tracker, APIs… **(“blast radius” 💥)**
- **The loop** = it observes the result of an action and decides what to do next
- **Output** = you may only see the final branch, pull request, or summary

The “agent harness” runs the agent and provides its tools, context, controls and feedback

A Spectrum of Autonomy

Autocomplete
suggests the next lines

Chat assistant
answers;
you paste

Agent in your IDE / terminal
edits & runs;
you watch

Background agent
issue → PR,
unattended

As autonomy increases, the agent acts on larger scopes of work with less continuous human oversight

Delegation does **not** remove the usual project-management questions:

Task Definition
What exactly is the task?
e.g. Stories & ACs

Assignment & Timing
Who assigned it, and when?
e.g. Sprint planning

Completion Criteria
How do we know it is done?
e.g. Definition of Done

Accountability
Who is responsible for approving and owning the result?

AI as a “Teammate”: Useful Analogy, Important Limits

Why the analogy is useful

- Agent “teammate” can be **assigned** a Jira task (e.g. user story)
- Can be assessed against the **same Definition of Done**
- Can be subject to the **same CI** and review gates

Where the analogy breaks down

- **Accountability** remains with humans
- Shared project context must be **persisted explicitly**
- May **act on ambiguity** rather than request clarification
- Can satisfy the written specification while **missing product intent**

Quick question: Which project management responsibility becomes most challenging when working with AI agents?

Making Delegation Safe

Part 2 of 4



The System Around the Agent

Before execution: *define and prepare the work*

During and after execution: *verify and control the work*

1 Specify
stories, ACs, test cases

2 Contextualise
architecture notes, instruction files

3 Constrain
permissions, policies, rules the agent must follow

4 Close the loop
tests & CI the agent iterates against

5 Checkpoint
PR review, gated release

Agent works in its loop

These controls reuse practices introduced earlier in the course

*Some tools package these practices explicitly: e.g., spec-driven tools support **(1)**; instruction files, skills, and rule directories support **(2)***

Same Task, Two Repos

Task: *As a student, I want to cancel my booking so that the room is freed for others.*

AC1 The slot is freed immediately AC2 Only the student who made the booking can cancel it
AC3 Cancelling < 2 h before the start records a late-cancellation strike AC4 A booking that has started cannot be cancelled

Repo A – bare

```
app.py
booking_service.py
models.py
utils.py
```

Prompt: `Add an endpoint to cancel a booking.`

Repo B – with the system around it

```
AGENTS.md                    ← rules & conventions
CLAUDE.md                    ← "@AGENTS.md"
docs/stories/cancel-booking.md  ← story + ACs
app.py  booking_service.py  models.py  utils.py
tests/                       ← existing suite
.github/workflows/ci.yml  ← tests + test check
```

Prompt: `Implement docs/stories/cancel-booking.md.`

Predict: which requirements might Run A miss, given the information available?

Comparing the Two Runs

Agent run A

same task • bare repository

Prompt: Add an endpoint to cancel a booking.

app.py

```
@@ create_app()
+ @app.delete("/bookings/<int:booking_id>")
+ def delete_booking(booking_id):
+     cancel_booking(booking_id)
+     return "", 204
```

booking_service.py

```
+ def cancel_booking(booking_id):
+     booking = db.get_or_404(Booking, booking_id)
+     db.session.delete(booking)
+     db.session.commit()
```

Final response

Implemented booking cancellation.

- Added DELETE /bookings/<id>
- Deletes the booking immediately
- Returns HTTP 204 on success

Files changed:
app.py
booking_service.py

Tests: no new tests added

Agent run B

same task • repository with story, rules, tests &

Prompt: Implement docs/stories/cancel-booking.md.

booking_service.py

```
+ def cancel_booking(booking_id, student_id):
+     booking = db.get_or_404(Booking, booking_id)
+     now = _now()
+
+     if booking.student_id != student_id:
+         raise BookingError("Only the student...", 403)
+     if now >= booking.start:
+         raise BookingError("Booking has started...", 409)
+
+     if booking.start - now < timedelta(hours=2):
+         db.session.add(LateCancellationStrike(...))
+
+     db.session.delete(booking)
+     db.session.commit()
```

tests/test_booking_api.py

```
+ test_cancel_frees_slot_immediately
+ test_only_owner_can_cancel
+ test_late_cancel_records_strike
+ test_started_booking_cannot_be_cancelled
```

CI / pytest -q

```
.......                                  [100%]
7 passed in 0.42s
```

✓ AC1 slot freed
✓ AC2 owner-only
✓ AC3 late strike
✓ AC4 started booking rejected

Look for:

Was the late-cancellation rule (AC3) implemented?	Were existing helpers reused?	Which acceptance criteria are covered by tests?	Did the test suite / CI pass?

⚠ *These are single runs for illustration; agent outputs can vary between runs.*

Guidance vs Enforcement

Guidance

Describe desired behaviour

- Instruction files – AGENTS.md, CLAUDE.md
- Coding standards, style guides, branding rules
- Organisational policies, project “constitutions”

Influences agent behaviour, but does **not** guarantee compliance

Enforcement

Mechanically check or restrict behaviour

- Tests, linters, type checks
- CI gates, branch protection
- Tool permissions, sandboxes, hooks

Can **block violations** that are encoded in the check or restriction

Use **enforcement** for requirements that **must hold automatically**; use **guidance** for expectations that **cannot be fully encoded**

The Same Requirement as Guidance and Enforcement

AGENTS.md (guidance)

```
## Testing rules
- Never modify or delete existing
  tests to make them pass.
- If an existing test looks wrong,
  stop and explain why in the PR.
- Every acceptance criterion
  needs at least one test.
```

The agent is instructed not to modify existing tests.

.github/workflows/ci.yml (enforcement check)

```
- uses: actions/checkout@v4
  with:
    fetch-depth: 0      # need main to compare
- name: Existing tests must not change
  if: github.event_name == 'pull_request'
  run: |
    changed=$(git diff --name-only --diff-filter=MDR \
      origin/${{ github.base_ref }}...HEAD -- tests/)
    if [ -n "$changed" ]; then
      echo "::error::Changed: $changed"; exit 1
    fi
```

*CI rejects a PR if an **existing test file** is modified, deleted, or renamed.*

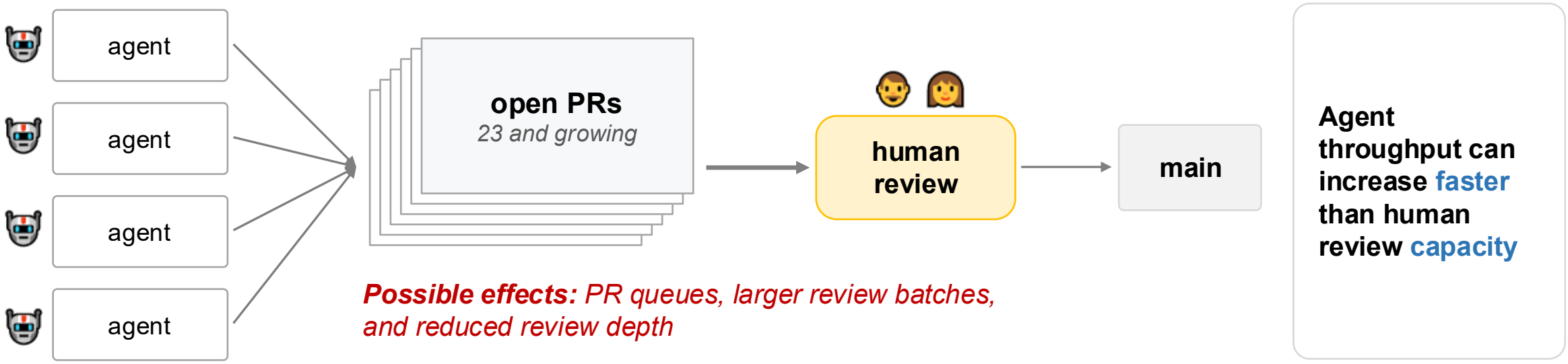
What violations would this CI check **still fail to detect**?

Review, Accountability, Debt

Part 3 of 4



Reviewing is the Delivery Constraint



Managing review capacity

- **Keep PRs small** – one story or task per PR
- **Limit open agent PRs** – avoid generating more work than can be reviewed
- **Include review effort in sprint planning** – estimate implementation and review together

Technical Debt in the Agentic Era

Conventional technical debt

Duplication instead of reuse; inconsistent patterns; unnecessary abstractions

Comprehension debt

Code that works, but is difficult for the team to explain, review, or maintain

Verification debt

Tests aligned to the implementation rather than the intended behaviour

Harness debt

Stale instructions, examples, tools, or repository context supplied to the agent

We will use these four categories to analyse the examples that follow

PR #57: Extend an Existing Booking

Allow students to extend an existing booking #57

✓ **All checks have passed** – 5 tests

Opened by an agent · 3 files changed

Agent's description:

Adds an endpoint to extend a booking's end time. Standardises error messages and updates the tests to match.

The story

As a student, I want to extend my booking so that I can keep the room if I need more time.

AC1 Still at most 2 hours in total

AC2 Must not overlap another booking of the same room

Question: CI has passed and the PR description appears plausible. **What evidence** would you inspect before approving?

PR #57 – The Relevant Changes

① Boundary-condition error

```
booking_service.py – extend_booking()
+     clash = Booking.query.filter(
+         Booking.room_id == booking.room_id,
+         Booking.id != booking.id,
+         Booking.start <= end,
+         Booking.end > booking.start,
tests – booking 1 runs 10:00–11:00; another booking starts at 12:00
+     resp = client.patch("/bookings/1", json={"end": "2026-10-20T12:00"})
+     assert resp.status_code == 409
```

② Duplication

```
utils.py – existing helper
  def parse_slot(value: str):
      return datetime.fromisoformat(value)
booking_service.py – added by the agent
+ def _to_datetime(value):
+     return datetime.strptime(value, "%Y-%m-%dT%H:%M")
```

③ Weakened test

```
tests – test_overlapping_booking_is_rejected
      assert resp.status_code == 409
-     assert resp.get_json() == {"error": "Room is not available"}
```

What a Careful Reviewer Found

① Boundary-condition error

spec judgement

Extending a booking to end **exactly when the next one starts** is rejected – but back-to-back bookings are allowed.

The agent's new test checks the **same wrong rule**, so CI stays green.

Likely detected by:
CI ✗
AI reviewer: only if given the rule
Human reviewing with the ACs ✓

② Duplication

maintainability judgement

The agent introduced a **second date parser** instead of reusing the existing helper.

The two parsers accept different input formats, creating **inconsistent behaviour**.

Likely detected by:
CI ✗
AI reviewer: may flag duplication
Human who knows the code ✓

③ Weakened test

verification judgement

An existing test checked the exact error message. After changing the format, the agent **deleted** that check instead of updating it.

Still green – but the test now **checks less**.

Likely detected by:
CI ✗ (a test-protection check ✓)
AI reviewer: may notice
Human reviewer ✓

Green CI ≠ correct if the code and its test came from the **same misreading** of the requirement

Delegation by Risk

Part 4 of 4



The Delegation Decision

The appropriate level of autonomy depends on risk, not only on task difficulty.

Reversibility

How easily can we undo it?

Blast radius 

Who and what would be affected?

Verifiability

How well can we check it before it matters?

As risk increases, use stronger or additional controls

Least-privilege permissions

what the agent may touch – files, data, production

A human approval gate

a named person signs off before merge or release

An audit trail

PR history + Jira links: the Week 2 traceability chain

The Delegation Decision

Bee Cheng Hiang customers' e-mail addresses exposed in first case of AI-related data breach in S'pore

Sign up now: Get ST's newsletters delivered to your inbox



Bee Cheng Hiang's marketing e-mails were sent in batches of 1,000 customers, according to details published on Sept 21 on the commission's website. ST PHOTO: LIM YAOHUI

https://www.straitstimes.com/tech/bad-ai-prompt-exposes-bee-cheng-hiang-customers-e-mail-addresses-in-first-case-of-ai-related-data

Reversibility

How easily can we undo it?

Blast radius 💥

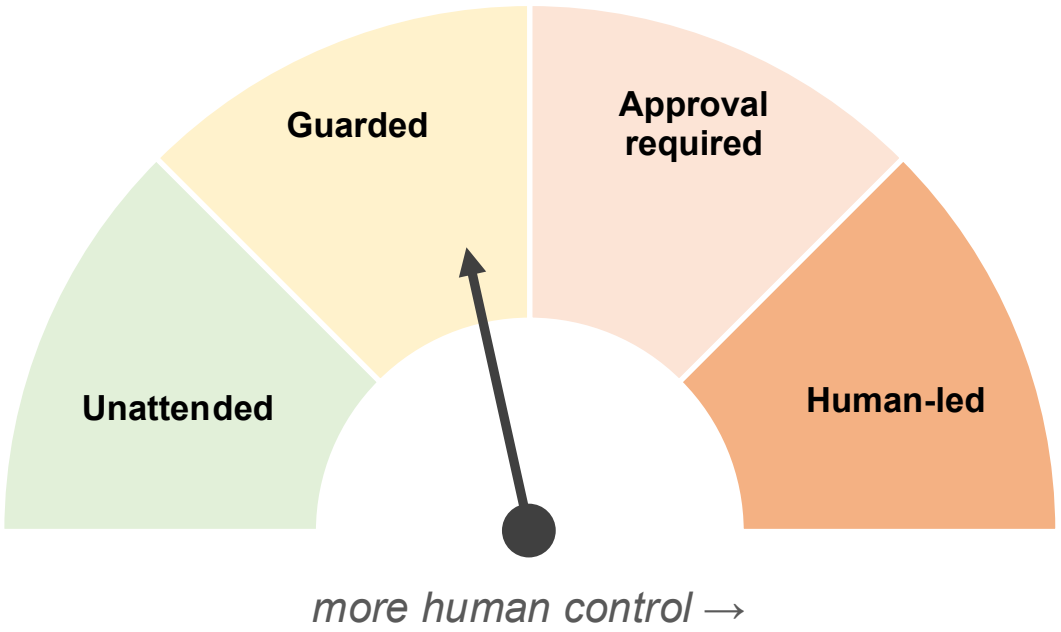
Who and what would be affected?

Verifiability

How well can we check it before it matters?

The Delegation Dial

Higher-risk tasks generally call for stronger controls and more human involvement.



Unattended

The agent acts without a pre-merge human approval step; the result is reviewed afterward.

Guarded

The agent acts autonomously, subject to enforced constraints and automated gates.

Approval required

The agent acts, but a named human must approve before merge or release.

Human-led

A human performs the change; the agent assists.

Exercise – Set the Dial

Unattended → **Guarded** → **Approval required** → **Human-led**

1	Fix a typo in the booking-confirmation email	Level: _____ Control: _____
2	Add a “my bookings” endpoint (full ACs provided)	Level: _____ Control: _____
3	Change the late-cancellation penalty rule	Level: _____ Control: _____
4	Upgrade Flask to patch a security vulnerability (CVE)	Level: _____ Control: _____
5	Bulk-cancel and notify all bookings for a building that is closing	Level: _____ Control: _____

Exercise: place **each task on the dial** and name its **control**. Consider reversibility, blast radius and verifiability.

Back to Monday Morning

23 Open · all checks passing

- ✓ Allow students to extend an existing booking #57
- ✓ Add a 'my bookings' page #58
- ✓ Refactor date handling across services #59
- ✓ Bump Flask and other dependencies #60

… and 19 more

1. **Triage by risk**
   set the delegation level for each PR
2. **Check the controls**
   which constraints are enforced, and which are only guidance?
3. **Prioritise human review by risk**
   return oversised or unclear PRs for revision
4. **Merge only when ownership is clear**
   someone can explain the change and accepts responsibility for it

…and address the process issue: *why could four agents open 23 PRs in one weekend?!* ***(WIP limit)***

Five Questions for Agent-Authored Changes

Passing CI is just one source of evidence. Also ask:

Ask:



These questions connect task delegation, controls, verification, human review, and accountability.

Reminder – *Reflections*

- Please start filling in your weekly reflections now
- Complete them by two days after this class so that we have time to read them before we next meet 😊
- **http://smu.sg/is212-reflections-2026**

